

ARQUITECTURA DE COMPUTADORES

Proyecto 2

Grupo 15:

Integrantes:

Martin Heiremans

Felix Svensson

Juan Pablo Saavedra

Profesor:

Jorge Gómez Mir

Santiago, 20 de Octubre de 2026

ÍNDICE

1. INTRODUCCIÓN **1**
2. DISEÑO GENERAL Y ARQUITECTURA **3**
Figura 1: Diagrama de bloques del sistema 3
2.1 Modificaciones al RTL 4
3. TEMPORIZACIÓN Y MEDICIÓN DEL TIEMPO **5**
3.1 Frecuencia de reloj y unidad de tiempo 5
3.2 Contador de ciclos y espera de tres segundos 5
3.3 Medición del tiempo de reacción 5
3.4 Conversión de ciclos a décimas 6
3.5 Cálculo del promedio 6
3.6 Selección pseudoaleatoria del LED objetivo 6
4. PROGRAMA EN ASSEMBLY **7**
Figura 2: Diagrama de flujo del programa 7
5. ASSEMBLER **8**
5.1 Sintaxis e instrucciones 8
5.2 Etiquetas e inmediatos 8
5.3 Validación y pruebas 8
5.4 Ejemplo de traducción 9

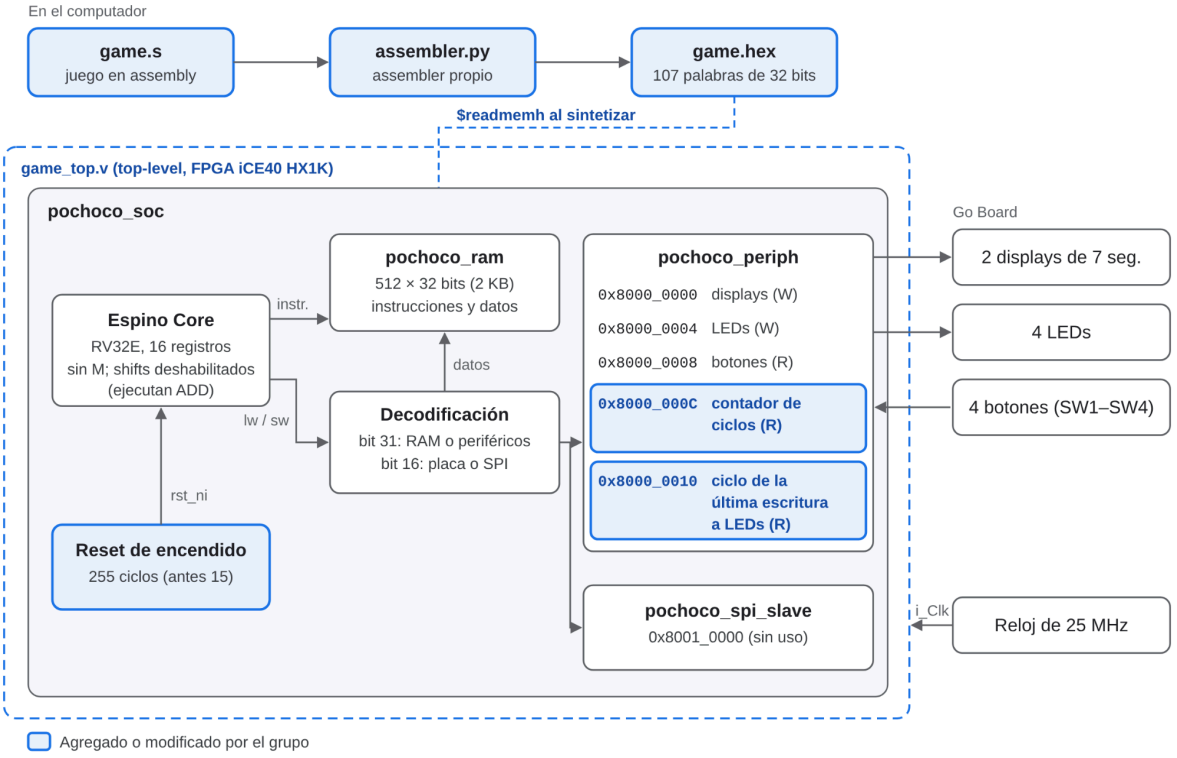
1. INTRODUCCIÓN

Este informe documenta un juego de reflejos escrito en assembly para el Espino Core, un procesador RV32E que se ejecuta dentro del Pochoco SoC sobre la FPGA Lattice iCE40 HX1K de la Go Board. El programa se traduce a código máquina con un assembler propio, y para medir tiempos se agregó al SoC un contador de ciclos que el programa puede leer. Como el procesador no tiene instrucciones de multiplicación ni división y sus desplazamientos están deshabilitados, todas las conversiones se hacen con sumas, restas y comparaciones.

2. DISEÑO GENERAL Y ARQUITECTURA

El sistema está formado por el Pochoco SoC y por un módulo superior propio, game_top, que lo instancia con la imagen sw/game.hex y conecta los pines de la Go Board. El Espino Core ejecuta el programa desde una RAM de 2 KB y accede a los LEDs, los botones y los displays mediante entrada y salida mapeada en memoria. Como cada instrucción tarda 2 o 3 ciclos, los tiempos no se miden contando instrucciones, sino con un contador de ciclos en hardware.

Figura 1: Diagrama de bloques del sistema



El programa accede a la memoria y a los periféricos mediante las siguientes direcciones:

Dirección	Acceso	Función
0x0000_0000 a 0x0000_07FF	Lectura y escritura	RAM unificada: instrucciones y datos
0x8000_0000	Escritura	Displays: bits 7–4 al izquierdo y bits 3–0 al derecho
0x8000_0004	Escritura	LEDs (4 bits)

0x8000_0008	Lectura	Botones (4 bits, 1 = presionado)
0x8000_000C	Lectura	Contador libre de ciclos (nuevo)
0x8000_0010	Lectura	Ciclo de la última escritura a los LEDs (nuevo)
0x8001_0000	Lectura y escritura	Esclavo SPI (sin uso en el juego)

2.1 Modificaciones al RTL

Se modificaron dos archivos del SoC base y se agregó el módulo superior; el Espino Core no se modificó. La tabla resume los cambios:

Archivo	**Cambio**	**Motivo**
pochoco_periph.v	Contador libre de 32 bits (cycle_q), legible en 0x8000_000C	Medir la espera de tres segundos y el tiempo de reacción, y obtener la semilla pseudoaleatoria
pochoco_periph.v	Registro led_cycle_q, que copia el contador en el mismo flanco en que se escriben los LEDs; legible en 0x8000_0010	Iniciar la medición en el mismo instante lógico en que se enciende el LED objetivo
pochoco_soc.v	Contador de reset de encendido de 4 a 8 bits: el SoC queda en reset 255 ciclos (unos 10 µs) en vez de 15	Dar un arranque limpio a la CPU y la RAM después de configurar la FPGA
game_top.v	Módulo superior nuevo	Instanciar el SoC con la imagen sw/game.hex y conectar los pines de la placa

3. TEMPORIZACIÓN Y MEDICIÓN DEL TIEMPO

3.1 Frecuencia de reloj y unidad de tiempo

La Go Board tiene un reloj de 25 MHz, por lo que cada ciclo dura 40 ns. Los tiempos se muestran en décimas de segundo, de 00 a 99 (hasta 9,9 s), y se convierten desde ciclos con las siguientes constantes:

Magnitud	Ciclos	Hexadecimal	Instrucciones
Una décima (0,1 s)	2.500.000	0x002625A0	lui x12, 0x262 y addi x12, x12, 1440
Divisor del promedio (10 × 0,1 s)	25.000.000	0x017D7840	lui x12, 0x17D8 y addi x12, x12, -1984
Espera inicial (3 s)	75.000.000	0x047868C0	lui x6, 0x4787 y addi x6, x6, -1856

Ninguna cabe en el inmediato de 12 bits de addi, así que se cargan con lui (20 bits altos) y addi (12 bits bajos). Como addi extiende el signo, si los 12 bits bajos valen 2048 o más se usa ese valor menos 4096 y la parte alta aumenta en uno.

3.2 Contador de ciclos y espera de tres segundos

El contador `cycle_q`, de 32 bits, avanza en cada ciclo de reloj. El programa solo usa diferencias entre dos lecturas, calculadas con sub y comparadas sin signo con bltu, que son correctas aunque el contador dé la vuelta entre ambas, siempre que pasen menos de 2^32 ciclos (≈ 171,8 s). Al comenzar cada ronda se encienden los cuatro LEDs, se lee el contador y se repite la lectura hasta que la diferencia alcanza 75.000.000 ciclos, de modo que los LEDs quedan encendidos al menos tres segundos.

3.3 Medición del tiempo de reacción

La medición comienza en el mismo flanco en que se enciende el LED objetivo: cuando `sw x7, 4(x1)` escribe los LEDs, el periférico copia el contador en `led_cycle_q`, y el programa lee ese valor con `lw x4, 16(x1)` como ciclo de inicio. Después lee los botones cada 6 ciclos de reloj y, al detectar una pulsación, calcula los ciclos de reacción como el contador actual menos el ciclo de inicio.

3.4 Conversión de ciclos a décimas

Como no hay instrucción de división, las décimas se obtienen restando 2.500.000 mientras el resto sea mayor o igual a ese valor y contando las restas, con un tope de 99. El resultado es la parte entera de la división:

```
décimas = ciclos / 2.500.000
```

Por ejemplo, 0,37 s son 9.250.000 ciclos y se muestran como 03. Como cada display muestra 4 bits en hexadecimal, el número se separa en decenas y unidades y se escribe decenas × 16 + unidades: 17 décimas se escriben como 0x17 y se ven como 17.

3.5 Cálculo del promedio

Cada respuesta correcta suma sus ciclos de reacción al acumulador x3; los intentos fallidos no se suman. Después de la décima ronda, el promedio en décimas, con división entera, es:

```
promedio = (x3 / 10) / 2.500.000 = x3 / 25.000.000
```

Se calcula con una sola división, por el mismo método de restas y con el mismo tope de 99, lo que evita acumular el error de truncamiento de cada ronda.

3.6 Selección pseudoaleatoria del LED objetivo

Después de la espera de tres segundos y de comprobar que no hay botones presionados, el programa calcula el objetivo así:

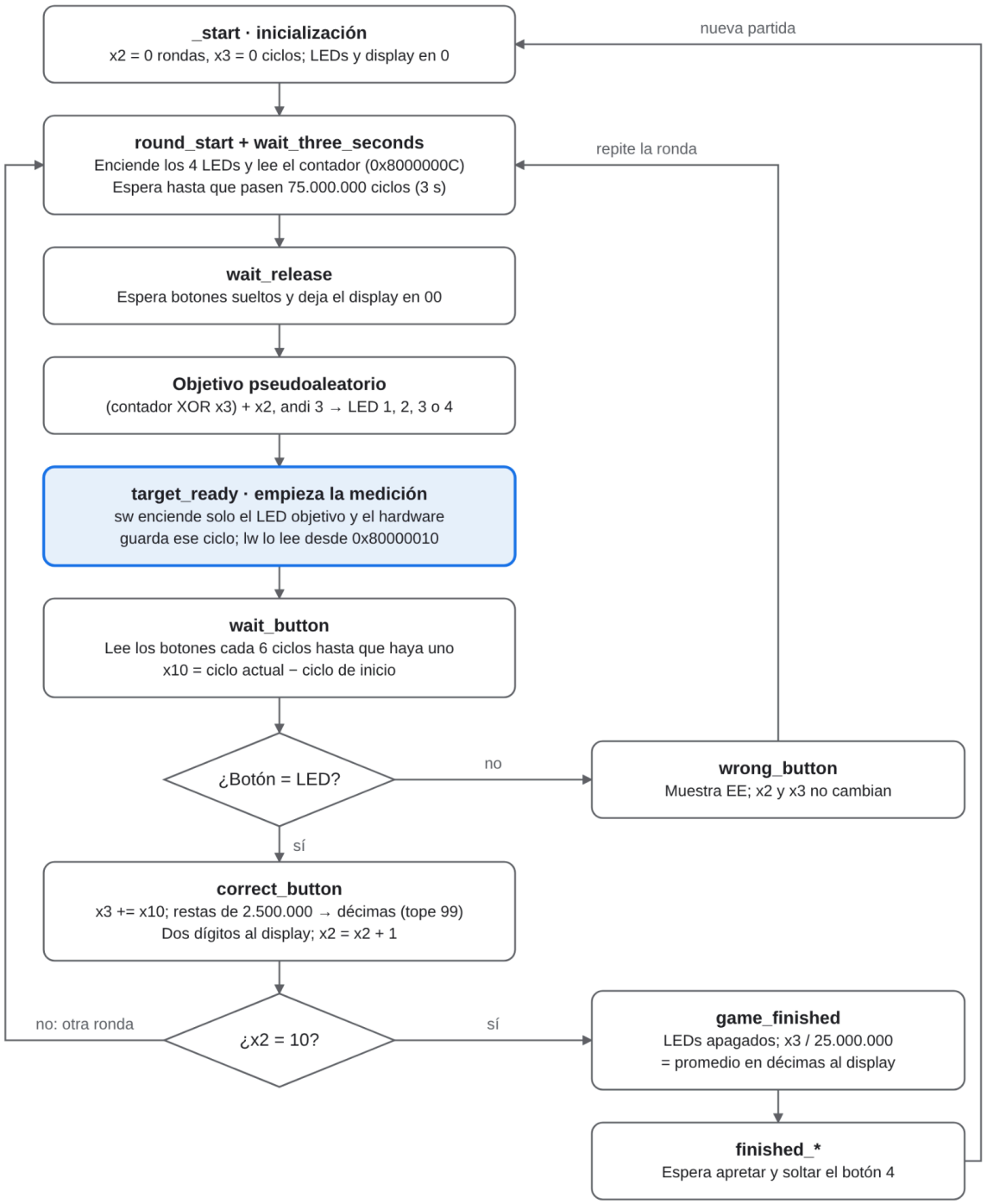
```
objetivo = ((contador XOR x3) + x2) AND 3
```

El valor del contador depende del instante exacto en que el jugador respondió la ronda anterior, con resolución de 40 ns, y x3 y x2 cambian en cada ronda, por lo que la secuencia no se repite entre partidas. Los dos bits resultantes se convierten en la máscara del LED (0001, 0010, 0100 o 1000) mediante comparaciones, porque no hay desplazamientos, y esa máscara se compara directamente con el botón leído.

4. PROGRAMA EN ASSEMBLY

El programa game.s tiene 107 instrucciones (428 bytes) y guarda todo su estado en registros, sin memoria de datos ni pila. La Figura 2 muestra su flujo.

Figura 2: Diagrama de flujo del programa



5. ASSEMBLER

El assembler, assembler.py, está escrito en Python y no usa herramientas externas. Recibe game.s y genera game.hex, con una palabra de 32 bits en hexadecimal por línea. Se ejecuta con `python3 assembler/assembler.py sw/game.s sw/game.hex`.

5.1 Sintaxis e instrucciones

Se escribe una instrucción por línea. Todo lo que sigue a # es comentario, las directivas como .section y .global se ignoran y una etiqueta es un nombre seguido de dos puntos. Los registros van de x0 a x15, sin nombres ABI; los inmediatos pueden ser decimales, hexadecimales (0x) o binarios (0b), y los accesos a memoria usan la forma `desplazamiento(registro)`, por ejemplo `4(x1)`.

Se aceptan las instrucciones de RV32I de los formatos U, I, R, S, B y J que el Espino ejecuta correctamente. Se rechazan las de desplazamiento (sll, srl, sra, slli, srli y srai), porque la ALU las ejecutaría como una suma, y tampoco se aceptan pseudoinstrucciones, mul, ecall ni fence. El juego usa 13 instrucciones: lui, addi, add, sub, xor, andi, lw, sw, beq, bne, bltu, bgeu y jal.

5.2 Etiquetas e inmediatos

El assembler hace dos pasadas. En la primera asigna a cada instrucción su dirección, de 4 en 4 desde 0, y a cada etiqueta la dirección de la instrucción siguiente. En la segunda codifica cada instrucción; en los saltos, el desplazamiento es la dirección de la etiqueta menos la de la instrucción, por lo que una etiqueta puede usarse antes de definirse.

Cada inmediato se valida contra el rango de su formato: de −2048 a 2047 en los formatos I y S, 13 bits con signo en los saltos condicionales y 21 bits con signo en jal, con desplazamientos siempre pares. En lui se usan los 20 bits bajos del valor, y las constantes más grandes se escriben como un par lui y addi (sección 3.1).

5.3 Validación y pruebas

Ante un registro inválido, un inmediato fuera de rango, un salto no alineado, una etiqueta repetida, una instrucción no soportada o un operando de memoria mal formado, el assembler se detiene sin escribir el archivo de salida, informa la causa y termina con código 1. El archivo test_assembler.py tiene seis pruebas automáticas, un programa válido comparado palabra por palabra y cinco casos de error, y todas pasan.

5.4 Ejemplo de traducción

La instrucción `bltu x5, x6, wait_three_seconds` está en la dirección 0x30 y su etiqueta en 0x28, así que el desplazamiento es −8, que en 13 bits es 1 1111 1111 1000. Con rs2 = x6 (00110), rs1 = x5 (00101), funct3 = 110 y el opcode de los saltos, 1100011, el formato B reparte los campos así:

Campo	**imm[12]**	**imm[10:5]**	**rs2**	**rs1**	**funct3**	**imm[4:1]**	**imm[11]**	**opcode**
Bits	1	111111	00110	00101	110	1100	1	1100011

Al concatenar los campos se obtiene 1111 1110 0110 0010 1110 1100 1110 0011 = 0xFE62ECE3, que es la línea 13 de game.hex.